PCB

Poornima Chandrashekhar Balagondar

AI & Machine Learning Engineer

PROFESSIONAL SUMMARY

AI & ML Engineer with hands-on experience building production-grade systems using LLMs, transformer models, and end-to-end ML pipelines. Delivered a multi-model fraud detection application integrating XGBoost and Llama 3.1 with natural-language explainability and a live Streamlit interface. Proficient in Python, NLP, Generative AI (RAG, LangChain, Agents), and data visualization. Passionate about scalable, data-driven solutions.

poornimabalagondar@gmail.com · +91 8310704458 · github.com/cbpoornima0511 · linkedin.com/in/poornima-chandrashekhar-balagondar

CAREER OBJECTIVE

Seeking an AI/ML or Generative AI engineering role where I can apply my expertise in LLM integration, prompt engineering, RAG pipelines, and production ML systems to build intelligent solutions that drive measurable business outcomes. Eager to contribute to a forward-thinking team and grow as an AI engineer solving complex, real-world challenges at the intersection of research and product.

Python · XGBoost · Llama 3.1 · LangChain · RAG · Streamlit · Scikit-learn · HuggingFace · Prompt Eng. · NLP

Generative AI Solutions
Google Cloud Skills Boost

Fundamentals of Artificial Intelligence
NPTEL

AI/ML Professional Development
Data Science with Generative AI

WHO I AM

B.E. Computer Engineering graduate (AI & ML) from Presidency University, Bengaluru (CGPA 7.88). I specialise in end-to-end ML pipelines, LLM integration, and prompt engineering — bridging the gap between raw model capabilities and real-world business impact. I enjoy building systems where ML predictions meet LLM reasoning, turning complex outputs into decisions people can trust and act on.

TECHNICAL SKILL MATRIX

Generative AI & Prompt Eng.

LLM Integration (Llama 3.1, GPT-2)	90%
Prompt Engineering (Zero/Few-shot, CoT)	88%
RAG & LangChain	83%
HuggingFace / Agents / Pydantic	80%

Machine Learning

XGBoost & Scikit-learn Pipelines	92%
Classification / Regression / Clustering	88%
Model Evaluation (F1, AUC-ROC)	86%
Cross-Validation & Grid Search	84%

NLP

Text Preprocessing & Vectorisation	85%
NER, POS Tagging, Word Embeddings	80%
Transformer Fine-tuning (RoBERTa)	78%
TF-IDF, BoW, Sequence Modelling	82%

Data Science & Dev Tools

Python (OOP, DSA, Decorators, Regex)	92%
EDA, Feature Engineering, Statistics	86%
Streamlit, Matplotlib, Seaborn, Power BI	83%
SQL / Pandas / NumPy / Excel	80%

TOOLS & TECHNOLOGIES

Python · SQL · MySQL · Scikit-learn · XGBoost · Streamlit · LangChain · HuggingFace · Git · VS Code · Jupyter · Power BI · Matplotlib · Seaborn · Pandas · NumPy

EDUCATION & EXPERIENCE

B.E. Computer Engineering (AI & ML)
Presidency University, Bengaluru
2021–2025 · 7.88 CGPA

PUC · Science
Anmol Science PU College, Davanagere
2019–2021 · 86.5%

SSLC
Roshni High School, Hangal, Haveri
2019 · 84.32%

Machine Learning Intern · InternPe — Jul 2024 · Aug 2024

Built end-to-end ML pipelines using scikit-learn (preprocessing, feature engineering, evaluation). Applied cross-validation & GridSearchCV, improving baseline accuracy by ~12%. Delivered model performance reports with actionable stakeholder insights.

Medical Query Classification & AI-Generated Guidance System

Stroke Risk Assessment · Logistic Regression · Llama 3.1 8B

PROBLEM STATEMENT

Stroke is a leading cause of death and disability worldwide, yet most people lack access to timely, personalised risk assessment. Traditional clinical screening is manual and resource-intensive. This project builds an AI-powered Stroke Risk Assessment Assistant that classifies patient stroke risk using a supervised ML model and augments it with Llama 3.1 8B — providing personalised, empathetic guidance in plain language.

BUSINESS OBJECTIVE

Enable non-clinical users to self-assess stroke risk, receive AI-generated health recommendations, and consult an intelligent chatbot for stroke-related queries — reducing preventable strokes through early awareness and actionable, personalised guidance.

SOLUTION

Component 1 — ML Classifier: Logistic Regression with Sklearn ColumnTransformer: StandardScaler for numerics (age, glucose, BMI), OneHotEncoder for categoricals (gender, work type, smoking status). class_weight='balanced' handles imbalance. Serialised to stroke_model.pkl.
Component 2 — LLM Layer: Llama 3.1 8B Instruct via HuggingFace Router — conversational medical AI answering stroke questions with built-in safety disclaimers.

KEY FEATURES

Risk Assessment Form
Age · Gender · Hypertension · Heart Disease · BMI · Glucose · Smoking

Risk Scoring Engine
Rule-augmented score → LOW / MEDIUM / HIGH with animated progress bar

AI Guidance Generator
Personalised recommendations: smoking cessation, BP monitoring, diet

Llama 3.1 Chat
Persistent session-state chatbot for stroke queries (st.chat_input)

Batch Analysis
CSV upload for bulk patient screening with downloadable results

Emergency Sidebar
FAST warning panel for immediate emergency symptoms with 911 guidance

TECHNOLOGY STACK

ML Model Logistic Regression	**Preprocessing** Sklearn Pipeline	**Scaler** StandardScaler	**Encoder** OneHotEncoder
LLM Llama 3.1 8B Instruct	**LLM API** HuggingFace Router	**Prompt** INST format / Role-based	**Interface** Streamlit

MY CONTRIBUTION

Designed full ML pipeline from CSV loading to stroke_model.pkl serialisation. Implemented predict_stroke_risk() with rule-augmented scoring. Engineered LLM system prompt with role definition and safety guardrails (INST format). Built combined Streamlit page: risk form + results + AI guidance + Llama chat. Developed batch analysis module and animated CSS glassmorphism UI.

Stroke Risk Assessment — System Architecture & AI Pipeline

AI PIPELINE FLOW

① Patient Input Form
Age · Gender · Hypertension · Heart Disease
BMI · Glucose · Smoking · Work Type · Residence

② Sklearn ColumnTransformer Pipeline
Numeric (AGE, glucose, BMI) → SimpleImputer(mean) → StandardScaler
Categorical (gender, work, smoking) → SimpleImputer(const) → OneHotEncoder

③ Logistic Regression Classifier
class_weight='balanced' · Binary: Stroke(1) / No Stroke(0)
Saved as stroke_model.pkl via Pickle

④ Risk Scoring & Guidance Engine
Rule-augmented risk score (0–1) → LOW / MEDIUM / HIGH
Personalised health recommendations per risk factor

⑤ Llama 3.1 8B (HuggingFace Router)
INST-format prompt · /v1/chat/completions · model: meta-llama/Llama-3.1-8B-Instruct:novita
Error handling: Timeout / 503 / 401 / 429 / ConnectionError

⑥ Streamlit Web Application
Home: Risk Form + Results + AI Guidance · Chat: Llama Q&A (session state)
Batch: CSV upload + bulk scoring · About: System info + disclaimer

FOLDER STRUCTURE

Medical-Query-Classification-and-AI-Generated-Guidance-System/

```
app1.py          # Streamlit app: UI, risk form, Llama 3.1 chat, batch analysis
model1.py        # ML training: Logistic Regression ColumnTransformer pipeline
stroke_model.pkl # Serialised Sklearn pipeline (production-ready)
all_data.csv     # Stroke dataset: features + binary target
```

LLM PROMPT ENGINEERING STRATEGY

Prompt Format: INST format: "<s>[INST] {system_prompt}\n\nUser Query: {query} [/INST]"

Role Definition: "medical AI assistant specialised in stroke risk assessment"

Behaviour Rules: identify symptoms · assess urgency · explain risk factors simply · ask clarifying questi

Safety Guardrails: never diagnose or prescribe · always recommend professional consultation for symptoms

Error Handling: Timeout / HTTPError (503, 401, 429) / ConnectionError — all return user-friendly fallbac

Stroke Risk Assessment — Full System Architecture Diagram

Logistic Regression · Llama 3.1 8B · Streamlit

Stroke Risk Assessment — Challenges, Results & Learnings

CHALLENGES & ENGINEERING SOLUTIONS

Class Imbalance in Stroke Data

Challenge: Stroke cases are rare (minority class) making naive classifiers biased toward 'No Stroke'.

Solution: Applied class_weight='balanced' in Logistic Regression to penalise misclassification of stroke cases.

LLM API Reliability

Challenge: HuggingFace Router returns 503 (loading), 401 (auth), 429 (rate limit), Timeout errors in production.

Solution: Implemented full exception handling for all HTTP error codes with specific user-friendly fallback messages.

UI Complexity

Challenge: Combining risk assessment form + AI chatbot on one page risked layout clutter and poor UX.

Solution: Used st.columns([2,1]) with glassmorphism CSS cards, animated gradients, and persistent session state.

LLM Safety in Medical Context

Challenge: Medical LLM must never diagnose, prescribe, or replace professional medical advice — liability risk.

Solution: System prompt explicitly defines guardrails: role, behaviour, never-diagnose rule, always-consult reminder.

RESULTS DELIVERED

ML Pipeline
Logistic Regression with Sklearn ColumnTransformer — numeric & categorical preprocessing

LLM Integration
Llama 3.1 8B via HuggingFace Router with INST-format role-based system prompt

Streamlit App
3-tab app (Home+Chat, Batch, About) with animated CSS glassmorphism and emergency sidebar

Safety Design
Emergency FAST panel, medical disclaimer, and explicit never-diagnose guardrail in every prompt

KEY LEARNINGS

- Designed LLM system prompts with explicit role, behaviour constraints, and hard safety guardrails.
- Implemented multi-exception LLM error handling (timeout, 503, 401, 429, ConnectionError) with fallbacks.
- Used st.session_state for persistent conversational memory across Streamlit chat turns.
- Built combined pages (risk form + chat) using st.columns with custom animated CSS.
- Applied class_weight='balanced' to Logistic Regression for imbalanced medical datasets.

FUTURE ENHANCEMENTS

SHAP Explainability
SHAP waterfall plots for feature-level prediction explanations

Fine-tuned LLM
Fine-tune Llama on stroke/medical terminology for domain precision

Docker Deployment
Containerise for one-command cloud deploy (AWS/GCP/Azure)

EHR Integration
Connect to Electronic Health Records for automated data ingestion

Intelligent NHIS Insurance Claim Fraud Detection Using LLM

XGBoost · Llama 3.1 8B · 4-Tab Streamlit · Dec 2025 – Feb 2026

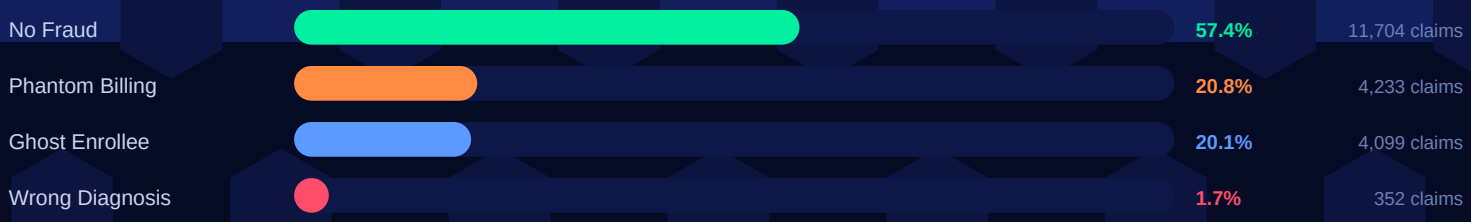
PROBLEM STATEMENT

Traditional NHIS fraud detection relies on manual audits and rigid rule-based checks — slow, expensive, and unable to keep pace with evolving schemes like Phantom Billing, Ghost Enrollees, and deliberate Wrong Diagnosis coding. Fraud analysts receive a flag but no explanation: they cannot act without understanding why a claim was flagged or what to investigate next.

DATASET OVERVIEW

20,388	7	4	80/20
Total NHIS Claims	Input Features	Fraud Classes	Train/Test Split

FRAUD CLASS DISTRIBUTION



SOLUTION

Layer 1 — XGBoost Classifier: Full Sklearn Pipeline with ColumnTransformer: numeric features (AGE, Amount Billed) → SimpleImputer(mean) + StandardScaler; categorical (GENDER, DIAGNOSIS, dates) → SimpleImputer(constant) + OrdinalEncoder(unknown_value=-1). LabelEncoder maps 4 fraud types. class_weight='balanced'. Serialised to model.pkl.

Layer 2 — Llama 3.1 8B: 4 distinct prompt templates (explanation / Q&A; / investigation report / decision recommendation) via HuggingFace Router /v1/chat/completions — making ML predictions auditable and actionable for non-technical fraud investigators.

KEY FEATURES

LLM-Powered Explanations
Llama 3.1 8B explains every prediction in plain English for analysts

Interactive Q&A
Ask claim-specific questions; LLM answers with full contextual awareness

Auto Investigation Report
One-click 5-section report: Summary·Classification·Risk·Patterns·Recs

Decision Recommendations
AI action plan: approve/escalate/verify + priority + dept + timeline

Probability Dashboard
Animated confidence bars for all 4 fraud types per analysed claim

Full Sklearn Pipeline
Imputation+encoding+scaling+XGBoost→model.pkl, zero drift at inference

MY CONTRIBUTION

Designed full ML pipeline from raw NHIS CSV to 4-class fraud prediction with model.pkl. Engineered XGBoost with ColumnTransformer for numeric + categorical features. Crafted 4 distinct LLM prompt templates (explanation, Q&A;, report, recommendation). Built 4-tab Streamlit app with animated CSS prediction cards colour-coded per fraud type. Implemented per-class probability bars and session-state claim context for cross-tab LLM queries.

NHIS Fraud Detection — System Architecture & AI Pipeline

END-TO-END AI PIPELINE

① Claim Input (Streamlit UI)
Patient ID · AGE · GENDER · DATE OF ENCOUNTER · DATE OF DISCHARGE · Amount Billed · DIAGNOSIS

↓

② Sklearn Preprocessing Pipeline (ColumnTransformer)
Numeric (AGE, Amount): SimpleImputer(mean) → StandardScaler
Categorical (GENDER, DIAGNOSIS, dates): SimpleImputer(const='unknown') → OrdinalEncoder(unknown=-1)

↓

③ XGBoost Multi-Class Classifier
LabelEncoder: No Fraud(0) · Phantom Billing(1) · Ghost Enrollee(2) · Wrong Diagnosis(3)
class_weight='balanced' · 80/20 split, random_state=42 · Serialised → model.pkl

↓

④ LLM Prompt Construction Layer
Claim data + predicted fraud type + confidence + per-class probabilities → 4 distinct prompt templates

↓

⑤ Llama 3.1 8B Instruct (HuggingFace Router)
POST https://router.huggingface.co/v1/chat/completions
model: meta-llama/Llama-3.1-8B-Instruct:novita · Auth: HF_TOKEN env var

↓

⑥ 4-Tab Streamlit Dashboard
Tab 1: Prediction + Confidence + Prob. Bars + AI Explanation
Tab 2: Interactive Q&A · Tab 3: Investigation Report · Tab 4: Decision Recommendation

FOLDER STRUCTURE

Intelligent-NHIS-Insurance-Claim-Fraud-Detection-Using-LLM/

```
NHIS Healthcare_claim_fraud.csv  # 20,388 NHIS claims (raw dataset)
model.py                         # Training: ColumnTransformer + XGBoost pipeline
model.pkl                        # Serialised Sklearn Pipeline (production-ready)
app.py                           # Streamlit 4-tab app + Llama 3.1 integration
README.md                        # Full documentation with architecture diagram
```

PROMPT ENGINEERING — 4 TEMPLATE SYSTEM

Tab 1 — Explanation: Role: fraud detection expert. Input: claim + prediction + probabilities → Plain English ex

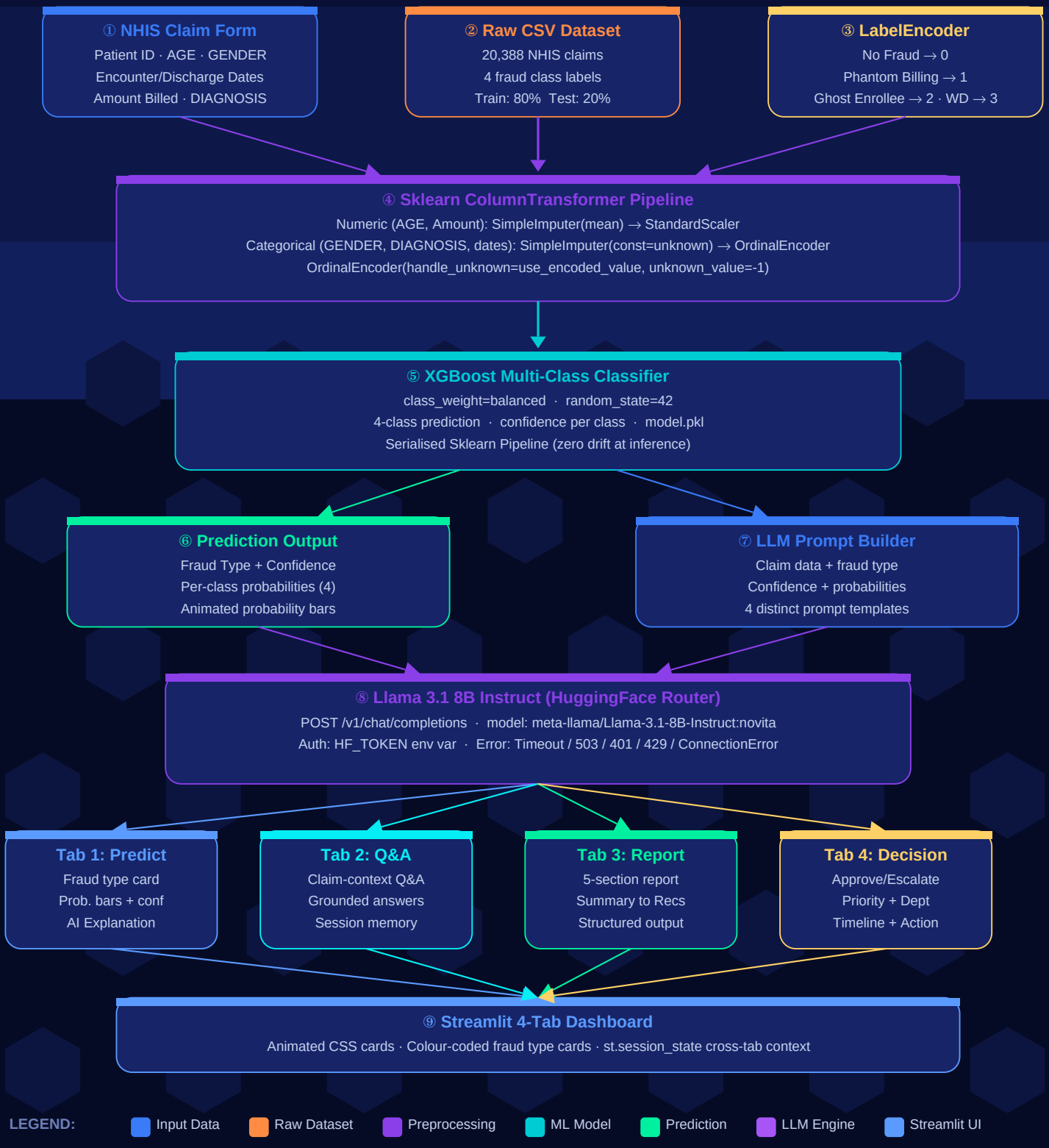
Tab 2 — Q&A: Role: fraud detection assistant. Input: question + claim context + probabilities → Grounde

Tab 3 — Report: Role: investigator. Input: full claim + prediction → 5-section structured report (summary/

Tab 4 — Decision: Role: analyst. Input: fraud type + confidence → Action plan (approve/escalate/verify + pri

NHIS Fraud Detection — Full System Architecture Diagram

XGBoost · Llama 3.1 8B · 4-Tab Streamlit



NHIS Fraud Detection — Challenges, Impact & Future Work

ENGINEERING CHALLENGES & SOLUTIONS

Wrong Diagnosis Class Imbalance (1.7%)

- **Issue:** Extreme class imbalance; naive classifiers ignore the minority Wrong Diagnosis class.
- **Fix:** class_weight='balanced' in XGBClassifier gives minority class appropriate training weight.

OrdinalEncoder — Unseen Diagnosis Codes at Inference

- **Issue:** New DIAGNOSIS strings at inference time cause encoder to crash with unknown category error.
- **Fix:** OrdinalEncoder(handle_unknown='use_encoded_value', unknown_value=-1) gracefully maps unknowns to -1.

LLM Report Consistency Across 4 Tabs

- **Issue:** 4 different LLM outputs (explanation, Q&A, report, rec.) need distinct structures and tones.
- **Fix:** 4 carefully crafted prompt templates with explicit output format instructions and role definitions per t

No Preprocessing Drift (Train → Serve)

- **Issue:** Applying different transformations at train vs. inference time causes silent performance degradation.
- **Fix:** Full sklearn Pipeline → model.pkl ensures identical ColumnTransformer applied at both training and servi

Missing Values in Real-World Claims

- **Issue:** Real NHIS data contains missing values in numeric and categorical feature columns.
- **Fix:** SimpleImputer(mean) for numerics, SimpleImputer(constant, 'unknown') for categoricals inside Pipeline.

MODEL DESIGN DECISIONS

class_weight='balanced'
Handles 1.7% Wrong Diagnosis without SMOTE

OrdinalEncoder unknown=-1
Robust to new diagnosis codes at inference

SimpleImputer in Pipeline
Graceful missing value handling in prod

Full Pipeline → model.pkl
Zero preprocessing drift train→serve

KEY LEARNINGS

- Engineered 4 distinct LLM prompt templates for explanation, Q&A, investigation report, and decision recommendation.
- Built multi-tab Streamlit apps with st.session_state for cross-tab LLM context persistence.
- Applied XGBoost with full Sklearn Pipeline ensuring reproducible, production-ready inference from model.pkl.
- Handled multi-class imbalance with class_weight and OrdinalEncoder unknown_value=-1 for robust inference.
- Designed animated CSS prediction cards colour-coded per fraud type and probability bars in Streamlit.

FUTURE IMPROVEMENTS & BUSINESS VALUE

SHAP Explainabil
Waterfall plots per ML prediction feature importance

Batch CSV Upload
Bulk claim screening with downloadable report

SMOTE Oversamp
Further address 1.7% Wrong Diagnosis class

Fine-tuned LLM
Llama fine-tuned on NHIS clinical terminology

Docker Deploymen
One-command containerised cloud deploy

Audit Trail
Persist all predictions + LLM responses for compliance

This production-ready architecture demonstrates the 'ML + LLM explainability' pattern standard in enterprise AI. Investigation reports reduce analyst time-to-action by surfacing structured risk factors and recommended verification steps — bridging ML predictions and investigator decisions.

GITHUB REPOSITORIES

Medical Query Classification & AI-Generated Guidance System

Stroke Risk · Logistic Regression · Llama 3.1 8B · Streamlit · Python

■ *github.com/cbpoornima0511/Medical-Query-Classification-and-AI-Generated-Guidance-System*

Files: app1.py | model1.py | stroke_model.pkl | all_data.csv

app1.py: Streamlit app + Llama chat · model1.py: ML training pipeline · stroke_model.pkl: Serialised model

Intelligent NHIS Insurance Claim Fraud Detection Using LLM

XGBoost · Llama 3.1 8B · 4-Tab Streamlit · 20,388 NHIS Claims · Python

■ *github.com/cbpoornima0511/Intelligent-NHIS-Insurance-Claim-Fraud-Detection-Using-LLM*

Files: app.py | model.py | model.pkl | NHIS...csv | README.md

app.py: 4-tab Streamlit + LLM · model.py: XGBoost training · model.pkl: Serialised pipeline

ARCHITECTURE DIAGRAMS (MERMAID)

■ **Project 1 — Stroke Risk System:**

```
graph LR: A[Patient Input] --> B[ColumnTransformer] --> C[LogisticRegression] --> D[Risk Score] --> E[Lla
```

■ **Project 2 — NHIS Fraud System:**

```
graph LR: A[Claim Input] --> B[Sklearn Pipeline] --> C[XGBoost] --> D[Prediction+Conf] --> E[4 Prompt Tem
```

SCREENSHOT CHECKLIST

Project 1 — Stroke Risk Assessment

- ■ Streamlit home page with risk assessment form (3-column layout)
- ■ Risk result card (LOW/MEDIUM/HIGH) with animated progress bar
- ■ Personalised AI guidance recommendations section
- ■ Llama 3.1 AI Chat interface with persistent session history
- ■ Batch Analysis page with CSV upload and downloadable results
- ■ Emergency FAST warning sidebar panel

Project 2 — NHIS Fraud Detection

- ■ Tab 1: Claim input form with animated fraud prediction card
- ■ Tab 1: Confidence score + probability bars (4 fraud types)
- ■ Tab 1: Llama 3.1 AI Explanation in plain English
- ■ Tab 2: Interactive Q&A with contextual LLM response
- ■ Tab 3: Auto-generated 5-section investigation report
- ■ Tab 4: Decision recommendation with priority & timeline

DEMO WALKTHROUGH SUMMARY

Project 1 Step 1: Open app → complete risk form (age, conditions, BMI, glucose, smoking) → click 'Analyze Risk'

Project 1 Step 2: View risk result card + progress bar + personalised AI guidance → switch to Chat tab

Project 2 Step 1: Enter NHIS claim details → click 'Analyze Claim' → view fraud type + confidence + probability b

Project 2 Step 2: Read Llama 3.1 explanation → switch to Q&A tab → ask follow-up questions about the claim

Project 2 Step 3: Go to Report tab → click 'Generate Investigation Report' → view 5-section structured report

Project 2 Step 4: Go to Decision tab → click 'Get Recommendation' → view action plan with priority and timeline

PCB

Thank You

for taking the time to review my portfolio

Poornima Chandrashekhar Balagondar

AI & Machine Learning Engineer · Bengaluru, India

poornimabalagondar@gmail.com

+91 8310704458

github.com/cbpoornima0511

linkedin.com/in/poornima-chandrashekhar-balagondar

✦ OPEN TO OPPORTUNITIES IN ✦

AI / ML Engineering · Generative AI · Prompt Engineering · NLP Engineering · Data Science · LLM Systems

> *I don't just integrate LLMs — I engineer the precision that makes them reliable.*